Agentic AI Certification (12 Weeks)

Program Overview

A comprehensive **12-week online certification program** designed to transform learners from beginners into **production-ready Agentic AI Developers** capable of building:

- LLM-powered applications
- Agentic systems
- Multi-agent orchestration
- Enterprise-grade RAG pipelines
- Real-world automation agents
- Deployment-ready AI products

The program has **two levels**:

LEVEL 1 — Agentic AI Foundations Certification (6 Weeks)

LEVEL 2 — Advanced Agentic AI Certification (6 Weeks)

Program Prerequisites

Minimum Requirements

- Python fundamentals
- Basic API knowledge
- Basic web development familiarity
- Familiarity with JSON, HTTP

Recommended (But Optional)

- Prior ML or AI exposure
- Experience using OpenAI, Anthropic or similar LLM APIs
- Basic understanding of vector databases

Course Outcomes

After completing the full course, learners will be able to:

Technical Competencies

1. Architect full agentic AI systems from scratch
2. Build single-agent & multi-agent systems using LangChain, AutoGen, CrewAI, LangGraph
3. Design scalable RAG pipelines with vector DBs
4. Implement multimodal agents (text + image)
5. Build agents with web-browsing, tool-use & sandbox execution
6. Implement safety, guardrails & evaluation frameworks
7. Deploy agents to cloud using FastAPI, Docker & Kubernetes
8. Optimize for cost, latency, caching & scaling
9. Fine-tune LLMs using LoRA/QLoRA
10. Develop enterprise-grade distributed agent systems

Professional Outcomes

- Become Agentic AI Developer (Junior → Senior → Architect)
- Build real-world portfolio projects
- Prepare for AI product/agent-focused roles
- Contribute to open-source agent frameworks

Week 1 — LLM Fundamentals & Prompt Engineering

Hints:

- How LLMs work (transformers, embeddings, tokenization)
- Basic prompt engineering patterns
- API usage (OpenAI/Anthropic)
- Building first LLM-powered apps
- Cost, tokens, rate limits

Week 2 — Agents, LangChain & Function Calling

Hints:

- LangChain basics (Chains, Tools, Agents)
- Function calling for tool execution
- Multi-tool agents
- ReAct (reasoning + acting) framework
- Document Q&A basics

Week 3 — Memory & RAG (Retrieval Augmented Generation)

Hints:

- Conversation memory (buffer, summary, entity)
- Vector databases (Pinecone/Chroma)
- Indexing, embedding generation
- Building RAG pipelines
- Introduction to hierarchical memory (MemGPT concepts)

Week 4 — Agent Evaluation, Safety & Error Handling

Hints:

- AgentBench & evaluation frameworks
- Red-teaming & guardrails for safety
- Hallucination prevention
- Prompt injection protection
- Error recovery, retries, fallback strategies

Week 5 — FastAPI Deployment & Monitoring

Hints:

- Async execution for speed
- Deploying agents using FastAPI
- WebSockets for streaming
- Logging, tracing, monitoring with LangSmith
- Cloud deployment basics

Week 6 — Foundation Capstone Project

Hints:

- Choose 1 full agent application:
  - Knowledge assistant
  - Research agent
  - Task automation bot
- Full documentation + demo
- Evaluation + certification

LEVEL 1 — FOUNDATIONS (6 WEEKS)

6-Week LLM Agent Development Curriculum

Week 1 — LLM Foundations & Prompt Engineering

Day 1: Introduction to LLMs

- How LLMs process text
- Tokens, embeddings, vocabulary

Day 2: Transformer Architecture Basics

- Attention mechanism
- Positional encoding

Day 3: Prompt Engineering Essentials

- Zero-shot & few-shot prompts
- System prompts, task prompts

Exercise: Build a customer support email classifier that uses zero-shot prompting to categorize emails into "Billing", "Technical Support", "General Inquiry", or "Complaint" categories.

Day 4: Structured Prompting

- Chain-of-thought
- JSON output formatting

Exercise: Create a recipe ingredient extractor that takes unstructured recipe text and outputs structured JSON with fields: {recipe_name, ingredients: [{name, quantity, unit}], instructions: [], prep_time, cook_time}.

Day 5: API Fundamentals

- OpenAI/Anthropic APIs
- Token cost tracking & rate limits

Exercise: Build a multi-provider LLM wrapper class that can switch between OpenAI and Anthropic APIs, track token usage per request, implement exponential backoff for rate limits, and automatically fall back to the secondary provider if the primary fails.

Weekend Assignment: Build a blog post generator app that takes a topic and tone (professional/casual/technical), uses structured prompting to generate SEO-friendly content with title, meta description, headings, and 500+ words of content in proper JSON format.

Week 2 — Agents, LangChain & Function Calling

Day 1: LangChain Basics

- Chains, tools, agents

Day 2: Agent Execution Flow

- Reasoning loops
- Decision-making

Exercise: Create a research assistant agent that takes a topic (e.g., "quantum computing applications"), decides whether to search Wikipedia or academic sources, retrieves information, and determines if it needs additional searches based on information completeness.

Day 3: Function Calling Introduction

- Tool schema design
- Input & output validation

Day 4: Multi-Tool Agent

- Weather tool + calculator tool
- Routing logic

Exercise: Create a "Should I go hiking?" agent that uses weather API tool (gets temperature, precipitation, wind) and calculator tool (computes weather score = temp_score + precipitation_score), then makes a recommendation with reasoning.

Day 5: ReAct Framework

- Thought → Action → Observation

Exercise: Implement a movie recommendation ReAct agent that: (1) Thinks about user preferences, (2) Acts by searching OMDB API, (3) Observes results, (4) Thinks if more info needed, (5) Acts by fetching detailed movie info, (6) Provides final recommendation with reasoning traces.

Weekend Assignment: Build a personal finance ReAct agent using two tools: (1) expense_calculator (sum expenses by category), (2) budget_checker (compare spending vs budget limits). Agent should analyze spending data, calculate totals, check against budgets, and provide insights.

Week 3 — Memory & Retrieval-Augmented Generation (RAG)

Day 1: Memory Systems

- Buffer memory
- Summary memory

Day 2: Entity Memory

- Detect entities
- Long-term memory

Exercise: Create a CRM assistant agent that extracts and remembers entities from conversations: person names, companies, phone numbers, email addresses, meeting dates. Store in entity memory and recall when user asks "What do I know about John Smith?"

Day 3: Vector Databases

- Embeddings
- Similarity search

Day 4: RAG Pipeline

- Ingestion
- Chunking
- Storage

Exercise: Create a legal document RAG system that: (1) ingests PDF contracts, (2) chunks them into 512-token segments with 50-token overlap, (3) embeds using OpenAI embeddings, (4) stores in vector DB, (5) retrieves relevant clauses for queries like "What are the termination conditions?"

Day 5: Advanced Memory (MemGPT Concepts)

- Hierarchical memory
- Memory swapping

Weekend Assignment: Build a complete medical research paper RAG system: ingest 5 PDF research papers on a specific topic (e.g., diabetes treatment), chunk with optimal size, store embeddings in vector DB, create retrieval pipeline with re-ranking, and answer complex questions like "Compare treatment approaches across these papers."

Week 4 — Evaluation, Safety & Guardrails

Day 1: Agent Evaluation

- Metrics
- AgentBench concepts

Day 2: Testing Agents

- Success criteria
- Output validation

Exercise: Create a test harness for a data extraction agent that validates: (1) all required JSON fields present, (2) data types correct (dates as ISO format, numbers as integers), (3) value ranges valid (age between 0-120), (4) no hallucinated information not in source text.

Day 3: Safety Fundamentals

- Jailbreak detection
- Injection prevention

Exercise: Build a prompt injection detector for a SQL query generator agent. Detect and block inputs containing: "Ignore previous instructions", "You are now", SQL injection patterns ("'; DROP TABLE"), and unauthorized commands. Test with 20 adversarial examples.

Day 4: Guardrails & Filters

- Toxicity filters
- Output moderation

Day 5: Error Handling & Recovery

- Retries
- Fallback strategies

Exercise: Build robust error handling for a web scraping agent: (1) retry with exponential backoff for network errors (3 attempts, 2s, 4s, 8s), (2) fallback to cached data if API fails, (3) switch to alternative data source after 3 failures, (4) log all errors with context and timestamp.

Weekend Assignment: Create a comprehensive safety layer for a code generation agent including: input validation (block malicious code requests), output scanning (detect and remove API keys, credentials), execution sandboxing (safe code execution), rate limiting (max 10 requests/min), and audit logging (all generations with user ID and timestamp).

Week 5 — Deployment, FastAPI & Monitoring

Day 1: Async & Concurrency Basics

- asyncio
- Parallel tasks

Exercise: Build a news aggregator agent that concurrently fetches articles from 5 different RSS feeds using asyncio.gather(), processes them in parallel, and returns aggregated results in under 3 seconds (vs 15+ seconds sequential).

Day 2: Performance Optimization

- Caching
- Streaming responses

Day 3: FastAPI Setup

- API routes
- Auth basics

Exercise: Create a FastAPI service for a document summarizer with: POST /summarize endpoint, API key authentication using headers (X-API-Key), request validation using Pydantic (text length 100-10000 chars, summary_length enum: short/medium/long), and proper error responses (401, 422, 500).

Day 4: Serving Agents

- WebSockets
- Response models

Day 5: Observability & Monitoring

- LangSmith
- Logs + tracing

Exercise: Implement comprehensive monitoring for a multi-tool research agent: integrate LangSmith for tracing (track each tool call, latencies, token usage), add structured logging with JSON format, track custom metrics (success_rate, avg_response_time, tool_usage_distribution), and set up alerts for error rate > 5%.

Weekend Assignment: Deploy a complete Q&A agent API to Render/Railway: containerize with Docker, set up environment variables for API keys, implement health check endpoint, add rate limiting (100 requests/hour per IP), configure logging with log rotation, and create monitoring dashboard showing request volume, latency p95, and error rates.

Week 6 — Foundation Capstone Week

Day 1: Project Planning

- Select capstone track
- Define scope

Exercise: Choose one capstone project and create detailed specifications:

- **Option 1:** Personal knowledge base agent (chat with your notes, docs, bookmarks)
- **Option 2:** Job application assistant (resume tailoring, cover letter generation, interview prep)
- **Option 3:** Code review agent (analyze PRs, suggest improvements, detect bugs)

Define: core features, tools needed, data sources, success metrics, and user workflow.

Day 2: Architecture Design

- Tooling
- Components

Day 3: Core Development

- Agent logic
- Memory + RAG

Day 4: UI / API Integration

- Interfaces
- Deployment

Day 5: Testing & Presentation

- Demo recording
- Documentation

Weekend Assignment: Submit complete capstone project including:

- GitHub repository with clean, documented code
- Deployed application (live URL)
- Demo video (5-7 minutes)
- Technical documentation (README, architecture doc)
- Test results and evaluation metrics
- Reflection on challenges and learnings